

SITAS

SYSTEM AND RUNTIME DOCUMENTATION

Frode Randers

2026-05-19

Contents

1	Introduction.....	3
1.1	Purpose	3
1.2	How to read this project	3
1.3	Scope of this manual.....	4
1.4	What this project is not	4
2	Architecture.....	5
2.1	Invariants	5
2.2	The shared-nothing rule	5
2.3	Boundaries and lifetimes	6
2.4	Layered model.....	6
2.5	Primary modules.....	7
2.6	How to trace a feature through the layers	7
3	Executor	8
3.1	Role	8
3.2	Responsibilities.....	8
3.3	Task lifecycle.....	8
3.4	Wakers and the ready queue	9
3.5	Scheduling groups	9
3.6	Structured child tasks.....	10
3.7	Wait ordering.....	10

3.8	Observability	10
4	Sharding.....	11
4.1	Shard-per-core direction.....	11
4.2	Cross-shard submission.....	11
4.3	Shard-local values	12
4.4	Sharded scheduling groups.....	12
4.5	CPU placement.....	12
5	I/O Runtime.....	14
5.1	Readiness.....	14
5.2	TCP helpers.....	14
5.3	io_uring	15
5.4	Completion lifecycle	15
5.5	Current limitation	16
6	Examples	17
6.1	Learning examples.....	17
6.2	Suggested reading path	17
6.3	Observability examples	17
6.4	Sharded index build	18
6.5	Representative commands.....	18
7	Operations	19
7.1	Building this manual	19

7.2 Cleaning generated files..... 19

7.3 Linux validation..... 19

7.4 Validation strategy..... 19

7.5 Interpreting unsupported features 20

1 Introduction

1.1 Purpose

`sitas` is an experimental Rust runtime and service model inspired by Seastar’s shard-per-core architecture. It is not a direct clone. The project is about understanding the runtime internals: how work is owned by shards, how cross-shard messages move, how a small executor drives futures, and how OS completion and readiness mechanisms can be exposed without depending on a third-party async runtime.

The code base started with a standard-library-only sharded key-value store. The current branch extends that baseline with a custom executor, Unix readiness primitives, TCP helpers, sharded async execution, shard-local values, observability snapshots, CPU placement, and experimental Linux `io_uring` support.

1.2 How to read this project

The easiest way to understand `sitas` is to read it from the outside in, then return from the inside out.

From the outside, the important idea is that application code should see typed, owned boundaries. A caller asks a service or a shard to do work. The runtime decides which shard owns that work. The owning shard mutates its local state and returns an owned reply. This is the same mental shape as a service call, but the implementation deliberately avoids hiding service state behind a global mutex.

From the inside, the interesting question is how the runtime keeps that model honest:

1. futures are owned by executor tasks;
2. tasks are assigned to one executor shard;
3. wakers only requeue tasks on that executor;
4. cross-shard work must be submitted explicitly;
5. shard-local values can be borrowed only inside synchronous closures;
6. observability returns copies, not borrowed internals.

This manual follows that path. The architecture chapter explains the ownership rules. The executor chapter explains how futures become runnable work. The sharding chapter explains how one executor becomes many independent shards. The I/O chapter explains why readiness and completion are different mechanisms and how both appear in the runtime.

Learning goal. If you can explain where a future is polled, who owns the state it mutates, what wakes it, and what happens when it is dropped, you understand the most important parts

of this code base.

1.3 Scope of this manual

This manual is a LaTeX companion to the Markdown architecture notes. It is intended for longer-form system documentation and for keeping a printable view of the evolving runtime design.

It focuses on:

- architectural invariants;
- executor and scheduling behavior;
- shard ownership and cross-shard submission;
- readiness, `epoll`, and `io_uring` integration;
- example programs that exercise the runtime;
- build and validation operations.

Note. The project is still deliberately experimental. APIs are allowed to move when that exposes cleaner runtime internals or a better teaching surface.

1.4 What this project is not

`sitas` is not trying to become a production replacement for mature Rust runtimes. It is also not trying to copy Seastar line by line. The point is to keep the mechanisms small enough that they can be inspected:

- no Tokio, `async-std`, `Glommio`, `Monoio`, or actor framework is used as the core runtime;
- blocking standard-library service APIs remain distinct from awaitable executor APIs;
- unsafe and FFI code is kept near the operating-system boundary;
- mechanisms are introduced only after their semantics can be tested.

This makes the project useful as a learning resource. It has enough machinery to demonstrate timers, wakers, readiness, `io_uring`, shard-local values, scheduling groups, and CPU placement, while still leaving those mechanisms visible in ordinary Rust modules.

2 Architecture

2.1 Invariants

The project is guided by a small set of invariants:

1. Each piece of application service state is owned by one shard.
2. Only the owning shard may mutate its service state.
3. Application services should not be modeled as shared `Arc<Mutex<Service>>` state.
4. Cross-shard interaction happens through typed messages, typed reply handles, or typed submitter calls.
5. Values crossing shard boundaries are owned.
6. References to shard-local state must not escape synchronous access closures and must not cross `.await`.
7. Async work remains on its assigned shard unless explicitly submitted elsewhere.
8. Observability snapshots are owned values, not borrowed runtime internals.
9. Unsafe code is isolated behind small safe APIs.

2.2 The shared-nothing rule

The central design rule is simple: application state has one owner. In the standard-library services that owner is a shard thread receiving typed commands. In the async runtime exploration that owner is an executor shard polling futures assigned to it. The form changes, but the rule is the same.

This is why the project avoids turning service state into `Arc<Mutex<Service>>`. A mutex would make mutation mechanically safe, but it would hide the runtime question that the project is trying to study: which shard owns this state, where does work run, and when does the ownership boundary get crossed?

The shared-nothing model does not mean the runtime never uses synchronization. It does. Wakers, reply state, lifecycle flags, counters, and submission queues are runtime machinery. The important distinction is that application service state is not normally protected by those locks. It stays local to its shard.

```
1 Caller thread or task
2   |
3   | owned command / owned future / typed submitter call
4   v
5 Owning shard
6   |
7   | synchronous local mutation
8   v
```

```
9 Owned reply or owned snapshot
```

Listing 2.1: State ownership model

2.3 Boundaries and lifetimes

Most runtime bugs in systems like this are boundary bugs. A value crosses to another thread without being owned. A reference to local state escapes a closure. A future keeps a borrowed value across `.await`. A completion operation drops a buffer before the kernel has finished with it.

`sitas` tries to make those boundaries visible:

Cross-shard transfer uses owned values and explicit `Send + 'static` requirements where a future can move to another thread.

Shard-local access gives a closure synchronous access to `&mut T`; the closure returns an owned result and cannot suspend.

Observability returns owned snapshots. A monitoring thread can keep a snapshot after the runtime has moved on.

Kernel I/O stores owned buffers until completions are observed or until abandoned operation cleanup proves they can be discarded.

2.4 Layered model

The current runtime can be read as a layered system:

```
1 Application service APIs
2 |
3 Typed command/reply and submitter APIs
4 |
5 Placement and shard routing
6 |
7 Shard-local service state / ShardLocal<T>
8 |
9 Shard executor or std shard thread
10 |
11 Mailbox, task queue, timers, readiness, wakeups
12 |
13 OS primitives: pipe/poll/epoll/io_uring/affinity
```

Listing 2.2: Conceptual layers

The standard-library baseline exercises the upper part of the model. The non-std runtime branch fills in the executor and OS layers while preserving the same shared-nothing service direction.

2.5 Primary modules

runtime The reusable standard-library kernel for bounded shard mailboxes, reply handles, shard startup, and shutdown.

kv The reference sharded key-value service.

counter A smaller second service proving that the runtime primitives are not key-value-specific.

placement Key-to-shard routing.

os Unix FFI, wake pipes, readiness backends, affinity, and Linux `io_uring` experiments.

executor The single-threaded async kernel.

sharded_executor One executor per shard thread, plus submission and observation APIs.

shard_local One owned value per executor shard with checked local access.

2.6 How to trace a feature through the layers

When learning or extending the runtime, trace one feature down the stack. For example, a named task in a scheduling group passes through several layers:

1. user code calls a typed spawn API;
2. the spawner validates that the scheduling group belongs to this executor;
3. a task is allocated with a name and group id;
4. the scheduler places the task into the group's ready queue;
5. the executor polls the selected task;
6. the task records status, wait interest, poll count, and timestamps;
7. a snapshot later resolves the group id back to an owned group name.

This kind of trace is more useful than reading modules in isolation. It shows which layer owns validation, which layer owns scheduling, which layer owns observability, and where the public API remains safe.

3 Executor

3.1 Role

The executor is a minimal single-threaded async kernel. It owns pinned futures inside tasks, maintains a ready queue, coalesces repeated wakes, and drives a root future through `run_until` while also polling spawned tasks.

The executor deliberately avoids Tokio or another third-party runtime. That keeps the interesting internals visible: wakers, task queues, timers, readiness registration, cancellation, and completion dispatch all live in this code base.

3.2 Responsibilities

- Poll ready tasks with a budget so self-waking tasks cannot starve timers and I/O forever.
- Implement custom task wakers that enqueue tasks onto the local ready queue.
- Catch task panics at executor boundaries.
- Preserve panic payloads where join handles and `block_on` make them observable.
- Provide `sleep`, `timeout`, `race`, and `yield_now`.
- Provide typed join handles and abort support.
- Provide cooperative stop tokens, `Notify`, and `TaskScope`.
- Drive readiness futures for read and write interests.
- On Linux, drive executor-owned `io_uring` read/write-at futures.

3.3 Task lifecycle

A spawned future is stored inside a task. The task has a stable executor-local `TaskId`, an optional name, a scheduling group id, and internal state describing whether it is queued, polling, waiting, completed, or cancelled.

The lifecycle is intentionally explicit:

```
1 spawn
2   -> create Task with pinned Future
3   -> queue Task in a scheduling group
4   -> poll Future
5       Ready(()) -> completed
6       Pending   -> waiting until waker fires
7   -> wake
8   -> queue Task again
```

Listing 3.1: Simplified task lifecycle

A repeated wake does not create repeated queue entries for the same task. The task records that it is already queued, so repeated wakeups coalesce. This is a small but important executor property: a chatty future should not multiply itself in the ready queue.

Cancellation is normal future dropping. Aborting a join handle or dropping an executor eventually drops the owned future. Timer and readiness futures must therefore clean up their registrations in `Drop`; otherwise the reactor would keep stale interests for tasks that no longer exist.

3.4 Wakers and the ready queue

The custom waker does one job: make the task runnable again on the same executor. It does not move the task to another shard, does not poll the task directly, and does not borrow application state. It marks the task queued and hands it back to the scheduler.

The executor loop then decides when to poll ready work. It does so in batches bounded by `READY_POLL_BUDGET`. That budget is a fairness valve. A self-waking task can keep itself ready, but it cannot prevent the executor from checking timers, readiness, and completions forever.

3.5 Scheduling groups

Scheduling groups are the first resource-class mechanism in the executor. Each group owns a ready queue and has a relative share count. Ordinary spawn calls use the default group. Explicit group APIs let code classify work such as foreground request handling and background maintenance.

The selection rule is deliberately simple. The scheduler chooses a non-empty group with the lowest weighted virtual runtime, pops one task from that group's FIFO queue, polls it, and charges the group for the observed wall-clock poll duration. The charge is scaled by the group's shares.

```
1 for each poll:
2     group = non_empty_group_with_lowest_virtual_runtime()
3     task = group.pop_front()
4     duration = poll(task)
5     group.virtual_runtime += duration * default_shares / group.shares
```

Listing 3.2: Weighted group selection

This is not preemption. A long poll still blocks the executor thread until the future returns `Poll::Pending` or `Poll::Ready`. The mechanism is cooperative: futures must yield, await I/O, await timers, or otherwise return control to the executor.

Scheduling group handles are executor-local. A group created by one executor is rejected by another. The default group handle is special because it names the built-in default queue in any executor. The same idea appears in the sharded runtime as a `ShardedSchedulingGroup`, which is one executor-local group per shard plus a runtime identity check.

3.6 Structured child tasks

`TaskScope` groups child tasks under one owner. Dropping the scope requests cooperative stop and aborts still-owned children. Calling `shutdown` asks children to stop and waits for them. Calling `shutdown_timeout` bounds that wait and aborts uncooperative children.

Scopes can also spawn children into scheduling groups. This matters when a server accepts work that should be both lifetime-scoped and resource-classified. The scope does not implement scheduling itself; it delegates group validation and task creation to the underlying `Spawner`.

3.7 Wait ordering

Each loop iteration polls ready work, wakes expired timers, dispatches locally available `io_uring` completions, and then decides how to wait.

Pending `io_uring` work now takes the executor wait slot when the shard has no ready tasks. That prevents unrelated timers or readiness interests from delaying completion-based file I/O. This is still not the final form: the long term goal is a single deadline-aware wait path that can combine timers, readiness, and completion waiting without giving any source accidental priority.

3.8 Observability

Executor snapshots are owned values. They include task counts, named task state, polling counters, timer counts, readiness counts, and Linux `io_uring` dispatcher counters when the dispatcher is installed.

Task snapshots include the task name, scheduling group id and name, lifecycle state, wait interest, poll count, accumulated poll time, and key timestamps. Helper methods derive task age and current state duration from a caller-supplied `Instant`. This keeps the snapshot itself factual while making progress views easy to write.

```

1 task name:           useful for humans
2 status:             queued, polling, waiting, completed, cancelled
3 waiting_for:        timer, readable fd, writable fd, unknown, none
4 poll_count:         how many times the future was polled
5 total_poll_time:    how much executor time the task consumed
6 state_duration_at(now): how long it has been in the current state

```

Listing 3.3: Reading a task snapshot

4 Sharding

4.1 Shard-per-core direction

`ShardedExecutor` connects the single-threaded executor to the shard-per-core model. It starts one executor on each shard thread and provides APIs for submitting work to one shard, all shards, or a map/reduce over shards.

The model is explicit: work does not migrate implicitly. A future runs on its assigned shard until code deliberately submits work elsewhere.

This mirrors the most important Seastar idea without copying its C++ API: parallelism comes from many mostly-independent shards, not from many threads mutating the same service object. Each shard has its own executor, task queues, timers, readiness registrations, and shard-local values.

1	Shard 0 thread	Shard 1 thread	Shard 2 thread
2	Executor	Executor	Executor
3	local tasks	local tasks	local tasks
4	local timers	local timers	local timers
5	local state	local state	local state

Listing 4.1: Shard-per-core runtime shape

4.2 Cross-shard submission

`ShardedSubmitter` is the explicit cross-shard async mechanism. A task on one shard can submit a future to another shard and await the returned join handle. The remote future is polled on the target shard. The awaiting task resumes on its original shard after the remote work completes.

Submitters own spawner clones. They are lifetime capabilities and must be dropped before a runtime can fully drain.

The important detail is the return path. A task on shard 0 can submit work to shard 1 and await the join handle. The submitted future is polled on shard 1. When it completes, the awaiting future on shard 0 is woken and continues on shard 0. The result crosses the boundary as an owned value.

```
1 Shard 0 task
2     submit future to Shard 1
3     await JoinHandle
4
5 Shard 1 executor
6     poll submitted future
7     complete owned result
8
9 Shard 0 task
10    wakes and receives result
```

Listing 4.2: Cross-shard async flow

This is why cross-shard APIs require `Send + 'static` where a future or value can move to another thread. It is also why references into shard-local state cannot be submitted.

4.3 Shard-local values

`ShardLocal<T>` owns one value per executor shard. Access is routed to the owning shard and the user closure receives `&mut T` synchronously. The implementation uses internal unchecked storage plus runtime owner checks, not a mutex around application state.

The critical rule is that references to local state cannot escape the closure and cannot cross `.await`. This keeps ownership local even when the runtime itself uses synchronization for task bookkeeping.

There are two useful ways to read `ShardLocal<T>`:

- as a safe teaching model for shared-nothing state, because each shard owns exactly one `T`;
- as a runtime boundary test, because the code must prevent references to `T` from becoming async state.

The API therefore prefers closures returning owned results. If a computation needs to wait, split it into two phases: first copy or compute an owned value from the shard-local state, then `await` using that owned value outside the access closure.

4.4 Sharded scheduling groups

A sharded scheduling group is not a global queue. It is a convenience handle containing one executor-local scheduling group per shard. Creating a group called `foreground` on a four-shard runtime creates four independent executor-local `foreground` groups, all with the same shares.

This preserves the shared-nothing model. Each shard still schedules only its own tasks. The sharded group handle simply lets callers use one typed API to spawn or submit matching classes of work across shards.

Runtime identity matters. A sharded group created by one `ShardedExecutor` is rejected by another runtime, even if both runtimes have a group with the same name. This prevents accidental targeting of same-numbered executor-local group ids in the wrong runtime.

4.5 CPU placement

CPU placement is an experimental Linux-supported runtime request. Linux applies hard affinity with `sched_setaffinity` where supported and observes container cpuset restrictions through `sched_getaffinity`. Non-Linux platforms report unsupported placement honestly.

By default, placement failures are recorded in shard snapshots and startup can still succeed. Required placement turns that into fail-fast startup behavior.

CPU placement is best understood as an explicit request, not as a finished portable scheduler. On Linux, the runtime asks the kernel to pin shard threads to particular CPUs. In containers, the available CPU set may already be restricted, so the runtime first observes what the kernel says is available. On macOS and other unsupported platforms, placement reports unsupported rather than pretending that pinning happened.

5 I/O Runtime

5.1 Readiness

The readiness path is intentionally small. A future registers interest in a file descriptor, the scheduler records the waker, the Unix reactor sleeps using Linux `epoll`, macOS/iOS `kqueue`, or fallback `poll` plus a wake pipe, and readiness wakes the interested task.

This path is readiness-based, not completion-based. It is used by the current TCP helpers and remains separate from the experimental `io_uring` completion path.

Readiness means that the operating system says an operation is likely to make progress. It does not mean the runtime has the bytes. A readable TCP stream may still return fewer bytes than requested. A writable stream may accept only part of a buffer. The future must attempt the operation, handle `WouldBlock`, and register interest again when necessary.

```
1 poll future:
2     try non-blocking operation
3     if operation succeeds:
4         return Ready(result)
5     if operation would block:
6         register fd interest and waker
7         return Pending
8
9 reactor wait:
10     OS reports fd readiness
11     scheduler wakes interested task
```

Listing 5.1: Readiness future shape

This is why readiness futures do not own file descriptors. The caller owns the stream or listener. The future only borrows it for the duration of the async operation and registers interest in the raw descriptor while pending.

5.2 TCP helpers

The TCP helpers are intentionally written on top of readiness futures rather than hiding a separate runtime. They demonstrate the normal async server shape:

1. put the listener or stream in non-blocking mode;
2. accept or connect using readiness futures;
3. spawn one handler task per accepted connection;
4. use `TaskScope` or `join handles` to wait for handlers;
5. propagate handler errors as ordinary `io::Result` values.

Grouped TCP helper variants classify accepted handler tasks into scheduling groups. The accept loop remains in the caller's task; only the per-connection handler work is moved into the requested group. This is useful for separating foreground request handling from background maintenance or low-priority connections.

Readiness lesson. Readiness is a notification that retrying may be useful. Completion is a notification that the operation has already finished. The code keeps those concepts separate because the ownership and cancellation rules are different.

5.3 `io_uring`

The Linux `io_uring` backend is experimental. It currently supports a narrow executor integration for owned-buffer `read_at`, `read_exact_at`, and `write_at` style file I/O.

Each Linux executor run loop installs a thread-local dispatcher when the host allows ring setup. Executor-backed `io_uring` futures store operation ids and owned buffers. When polled on a shard, they queue an operation against the thread-local dispatcher and register their task waker.

The dispatcher tracks:

- queued submissions;
- locally buffered completions;
- registered wakers;
- abandoned operations;
- deferred owned buffers;
- cumulative dispatched, buffered, woken, and discarded operation counters.

5.4 Completion lifecycle

The `io_uring` path is completion-based. A future submits an operation and later receives a completion. That changes the ownership problem. With readiness, the future borrows a stream and retries ordinary system calls. With `io_uring`, the kernel may hold a pointer to an owned buffer until the completion arrives.

The runtime therefore tracks operation state explicitly:

```
1 future first poll
2   allocate operation id
3   move owned buffer into dispatcher tracking
4   submit operation to ring
5   register task waker
6   return Pending
7
8 completion arrives
9   record result and completed buffer
```

```
10     wake registered task
11
12 future next poll
13     take completion
14     return Ready(result, buffer)
```

Listing 5.2: Simplified completion lifecycle

If the future is dropped before completion, the operation may still exist in the kernel. The dispatcher marks it abandoned and defers buffer cleanup until a completion or safe discard point is observed. This is one of the most important differences between readiness and completion: cancellation is not just removing a waker; it also involves kernel-owned in-flight work.

5.5 Current limitation

Timers, readiness waits, and `io_uring` waits are not yet unified into one production event source with deadline-aware completion waiting. The current executor gives pending `io_uring` work priority when there are no ready tasks, which is useful and test-covered, but it is still a staged integration.

6 Examples

6.1 Learning examples

The `examples/` directory is part of the teaching surface. The examples show the runtime pieces in isolation before combining them:

- basic executor futures and timers;
- readiness-driven TCP helpers;
- sharded executor startup and submission;
- shard-local values;
- CPU placement reporting;
- observability snapshots;
- standard-file and `io_uring` index-building demos.

6.2 Suggested reading path

The examples are easiest to learn from in layers:

1. Start with `executor_sleep.rs`, `executor_timeout.rs`, and `executor_race.rs`. These show futures, timers, and combinators before sharding enters the picture.
2. Move to `async_readable.rs`, `async_write.rs`, and `async_tcp_pair.rs`. These show readiness and non-blocking file descriptor behavior.
3. Read `async_tcp_server.rs`, `async_tcp_scoped_server.rs`, and `async_tcp_scheduling_groups.rs`. These show handler spawning, scoped shutdown, and scheduling groups.
4. Then read `sharded_executor.rs`, `sharded_submit.rs`, and `sharded_map_reduce.rs`. These show explicit placement of async work on shards.
5. Finally read `shard_local.rs` and `shard_local_worker_observability.rs`. These show local state access and owned runtime snapshots.

Treat examples as executable documentation. Run them, then inspect the modules they use. Most examples intentionally print runtime counters or snapshots so that the mechanism is visible rather than merely implied.

6.3 Observability examples

`sharded_observability.rs` samples task state while named shard tasks sleep and wake. The interesting columns are:

status the coarse task lifecycle state;
age_ms how long the task has existed;
state_ms how long it has been in the current state;
polls how many times the future has been polled;
wait the last known wait interest.

`async_tcp_scheduling_groups.rs` shows a different form of observability. It starts two TCP accept loops, parks accepted handlers, takes a snapshot, and prints which scheduling group each live task belongs to. This is a small substitute for a console UI: the snapshot is just owned data, so a tool can render it however it wants.

6.4 Sharded index build

The sharded index example uses two files: a fixed-size record data file and a sorted index of offsets into that data file. Each shard scans one partition, sorts its partition-local index entries, writes a materialized run file, and then participates in merge rounds until one sorted index remains.

The `io_uring` variant performs partition reads, run writes, and merge run reads/writes through the executor-backed Linux completion path. Final verification still uses ordinary file I/O because it checks the externally visible result rather than the runtime path that produced it.

The index examples are intentionally more involved than the small teaching examples. They exercise the Seastar-like direction: split work by shard, keep partition-local state local while sorting, materialize intermediate runs, and merge owned outputs. They are useful for studying where the project is heading, but they are not the first examples to read.

6.5 Representative commands

```
1 cargo fmt --check
2 cargo clippy --all-targets -- -D warnings
3 cargo test
4 cargo doc --no-deps
```

Listing 6.1: Local validation

```
1 SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo test
2 SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example
   sharded_index_build_uring -- --records 512 --shards 4 --seed 0xbeef --
   cleanup
```

Listing 6.2: Linux validation through Docker

7 Operations

7.1 Building this manual

The manual lives in `docs/system`. Build it with:

```
1 cd docs/system
2 make
```

The output is `docs/system/sysdoc.pdf`. The build uses `pdflatex` twice so the table of contents and references settle.

If the build fails, first check that the LaTeX distribution has the standard packages used by `sitasdoc.cls`: `geometry`, `fancyhdr`, `titlesec`, `titletoc`, `listings`, `xcolor`, and `hyperref`. The manual intentionally avoids project-specific generated assets so it can be rebuilt from text alone.

7.2 Cleaning generated files

```
1 cd docs/system
2 make clean
3 make distclean
```

`make clean` removes LaTeX auxiliary files. `make distclean` also removes the generated PDF.

7.3 Linux validation

Some runtime behavior is Linux-specific. The Docker helper mounts the project directory into a Linux container and can request an `io_uring`-capable configuration:

```
1 SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo test os::uring::tests --
  --nocapture
```

When the host or container configuration cannot provide `io_uring`, the Linux-only examples report that the backend is unavailable rather than faking a successful run.

7.4 Validation strategy

Use narrow tests first, then broaden. This keeps iteration fast while still protecting the runtime invariants.

```
1 cargo test task_snapshot -- --nocapture
2 cargo test scheduling_group -- --nocapture
3 cargo test serve_tcp -- --nocapture
4 cargo fmt --check
5 cargo test
6 cargo clippy --all-targets -- -D warnings
7 cargo doc --no-deps
```

Listing 7.1: Typical focused-to-broad workflow

For changes touching Linux-only code, use the Docker helper after local macOS validation:

```
1 SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo test
2 SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo clippy --all-targets -- -D
  warnings
```

Listing 7.2: Linux platform pass

7.5 Interpreting unsupported features

Unsupported is a valid result when the platform cannot provide a feature. CPU placement is unsupported on non-Linux platforms. `io_uring` may be unavailable if the host kernel, container runtime, or security profile blocks ring setup. The project should report those cases honestly instead of hiding them behind a fake success path.

This matters for learning. A runtime experiment should make platform boundaries visible: readiness works on several Unix backends, hard CPU affinity is Linux-specific, and `io_uring` is both Linux-specific and dependent on host/container permissions.